Network Science

Lecture 04: Communities and Graph Partitioning

Prof. Dr. Michael Granitzer

Dr. Kanishka Ghosh Dastidar

Dr. Jelena Mitrovic

Speaker notes

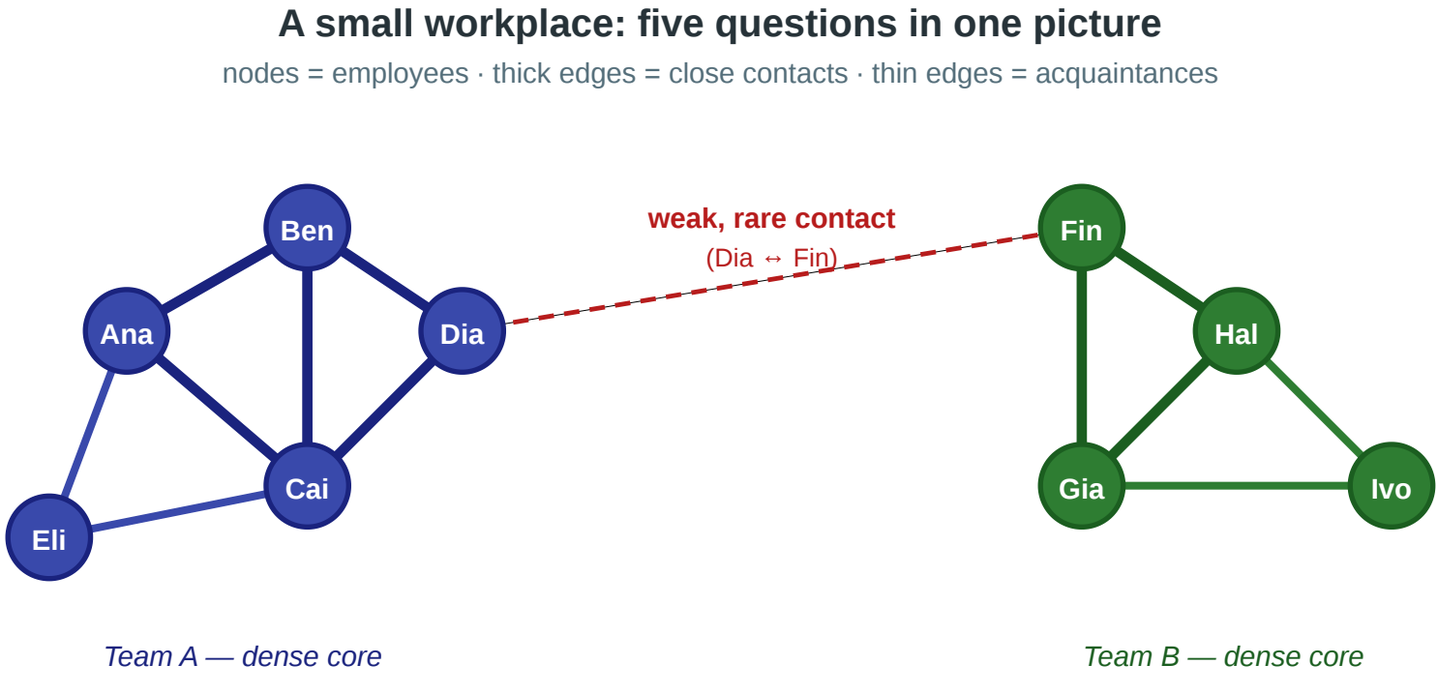
This lecture continues the modeling-vs-measurement arc from Lecture 03. In L03 we classified *edges* (strong vs. weak), discovered that the exact recovery problem (MaxSTC) is NP-hard, and fell back on polynomial-time structural proxies. In L04 we classify *nodes* (structural roles, centralities) and *sets of nodes* (communities), and we find exactly the same pattern: the theoretically ideal objective (modularity maximization) is NP-hard, and we fall back on heuristics. The whole course is held together by that single loop: theoretical construct → intractable recovery → polynomial-time proxy → empirical test.

From Classifying Edges to Classifying Nodes and Groups

In Lecture 03 we asked which *edges* carried novel information — strong vs. weak, bridges vs. embedded. Today we ask which *nodes* occupy important positions, and how the graph splits into dense groups.

Let us return to the same workplace.

The Same Scenario, New Questions



Questions we'd like to answer from this picture

① Will Eli and Ben become friends? ② How do we separate close from distant ties?
③ Which single edge controls all information flow between teams? ④ Where would a rumor spread fastest? ⑤ Which tie would we cut to isolate a leak?

Same graph as last time — two teams linked by a single weak Dia–Fin contact. Lecture 03 asked about the edges. Today the questions are about the nodes and the groups.

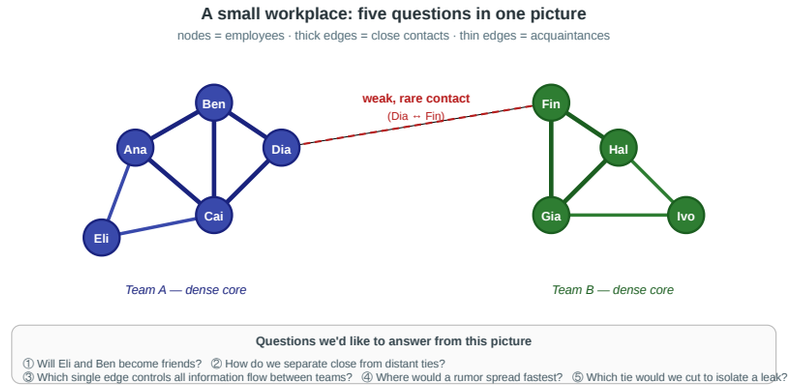
Speaker notes

Reusing the L03 scenario is deliberate. Students should feel that the lecture is continuing a single investigation rather than starting a new topic. The same figure will now reveal a different set of concepts depending on which question we ask.

Four Group-Level Questions

1. **Importance.** Who is the single most important person in this office — and what does "important" even mean?
2. **Boundaries.** Where is the line between team A and team B, and could an algorithm discover it *without being told the labels*?
3. **Uniqueness.** If we ran community detection on a 10 000-person company, would the answer be unique — or would different methods give different maps?
4. **Validation.** How do we know when a detected community is real rather than an artifact of the algorithm?

Every concept in today's lecture is a formal answer to one of these questions.



Speaker notes

Each question maps to a module: (1) structural roles and centrality, (2) communities and modularity, (3) divisive vs. agglomerative community-finding heuristics, (4) the Zachary karate club as an empirical anchor. Keep referring back to the workplace figure: Ana and Ben anchor the embedded role, Dia anchors brokerage, and the Dia–Fin edge anchors the community boundary.

Theory vs. Measurement — Continued

The gap we opened in Lecture 03 does not close in Lecture 04. Every concept today has a theoretical form we cannot compute on real data and a measurable proxy we can.

Question	Theoretical construct	Measurable proxy
Who is most important?	*(no single answer — multiple theories)*	Degree / closeness / betweenness / eigenvector
What makes a group a group?	Community with high internal density	Modularity Q
Best partition?	Global modularity maximum (NP-hard)	Girvan–Newman / Louvain heuristics
Is a partition meaningful?	Agreement with ground-truth labels	Benchmark datasets (Zachary's karate club)

Speaker notes

This table is L04's version of the L03 roadmap. The first row is slightly different from the others because centrality is not a single construct with a tractable/intractable split — it is a *family* of competing operationalizations, and the "measurement" column already reflects the theoretical pluralism. The second and third rows show the classical modeling/measurement split from L03, now applied to groups instead of edges.

Takeaway

L03 classified edges; L04 classifies nodes and groups. The modeling–measurement loop is the same: a theoretically clean objective is NP-hard, polynomial-time heuristics approximate it, and empirical benchmarks test whether the heuristics recover meaningful structure.

Speaker notes

This takeaway names the shape of the lecture before any content arrives. Refer back to it when introducing modularity ("this is the theoretical objective, and it is intractable to maximize exactly"), when introducing Girvan–Newman and Louvain ("these are the polynomial-time heuristics"), and when introducing Zachary's club ("this is the empirical test").

Roles and Social Capital

Guiding Question: What kinds of structurally different positions can people occupy inside the same network?

Speaker notes

The point of this module is to dismantle the intuitive notion that "important nodes" form a single category. A high-degree hub, an embedded clique member, and a structural broker are three distinct positions with different social and informational consequences. Students should leave the module able to name and contrast all three on the workplace figure.

Three Structural Positions

Embeddedness. A node whose neighbors are themselves densely interconnected. High clustering coefficient.

Brokerage. A node connecting otherwise separated groups. Low clustering coefficient, high betweenness.

Structural hole. The empty region in the network that a brokerage position spans.

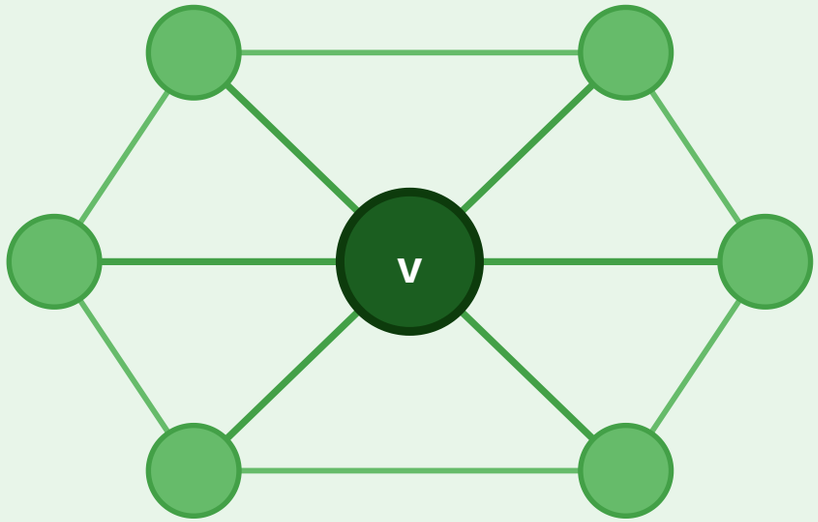
Speaker notes

Embeddedness is a property of a node, brokerage is a relational role, and a structural hole is the absence of edges that makes brokerage possible. In the workplace figure, Ana is embedded (her neighbors Ben and Cai are directly connected), Dia is a broker (the single cross-team tie runs through her), and the absence of direct Eli–Fin, Cai–Gia, Ben–Hal edges forms the structural hole between the teams.

Embedded vs. Broker

Embedded Node

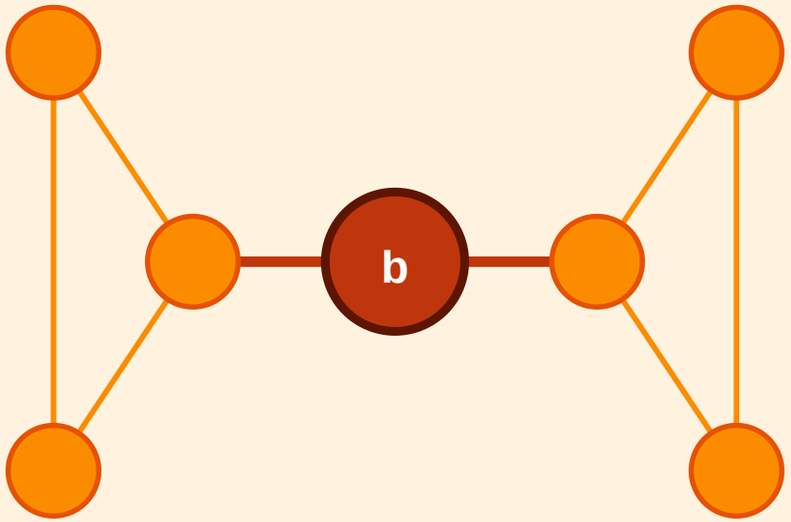
Dense, mutually connected neighbourhood



High clustering · trust · redundant info

Broker (Structural Hole)

Sole link between two separated clusters



Low clustering · high betweenness · novel info

The embedded node v sits inside a tightly connected neighborhood — its removal barely affects connectivity. The broker b is the only short link between two clusters — its removal would isolate them.

Speaker notes

The visual contrast highlights how the same graph structure produces two opposite kinds of importance. The embedded node enjoys trust, redundancy, and reinforced norms; the broker enjoys early information access and gatekeeping power. Burt's empirical research showed that brokers are systematically over-rewarded in organizational settings — they earn more, get promoted faster, and produce more innovations — precisely because they capture the value of structural holes.

Takeaway

"Importance" is not one thing. Brokerage, embeddedness, and degree capture different social advantages — and the same node can sit in multiple roles depending on the question.

Quick Check

Look again at the workplace figure (Ana, Ben, Cai, Dia, Eli on the left; Fin, Gia, Hal, Ivo on the right).

1. Which of these nine people is the clearest *broker*, and what structural feature makes them one?
2. Which is the most clearly *embedded*?
3. What would change structurally if Dia left the company tomorrow?

Speaker notes

Dia is the only broker: she is the only endpoint of the single cross-team edge. Ana, Ben, and Cai are all embedded inside the left triangle — each of them has directly connected neighbors. If Dia leaves, the only cross-team tie vanishes and the two teams become disconnected components — an extreme form of the structural-hole intuition. This quick check directly reuses the L03 scenario to make the new vocabulary concrete.

Quick Check — Solution

1. Broker: Dia — she is the only endpoint of the cross-team edge. Every shortest path between the two teams passes through her.

2. Embedded: Ana, Ben, and Cai are all embedded (they sit inside the same closed triangle). Ben is perhaps the most embedded, with all three neighbors (Ana, Cai, Dia) mutually connected.

3. If Dia leaves: the Dia–Fin edge disappears with her, and the two teams become *disconnected components*. The structural hole becomes absolute — no shortest path exists between Ana and Hal at all.

Centrality

Guiding Question: How should we measure which nodes are important?

Speaker notes

Centrality is not a single quantity but a family of operationalizations, each formalizing a different intuition about importance. The choice of centrality measure is a modeling choice with consequences — picking degree when betweenness is the right concept will systematically miss the most structurally critical nodes.

Centrality

Guiding Question: How should we measure which nodes are important?

$$\text{importance} \neq \text{degree alone}$$

Degree and Closeness Centrality

Degree centrality

$$C_D(v) = \frac{\deg(v)}{n-1}$$

Measures **local exposure** — how many direct contacts a node has. Fast to compute; blind to network position.

Example: The most-followed account on a social platform ranks high on degree — but "famous" and "structurally central" are different things.

Closeness centrality

$$C_C(v) = \frac{n-1}{\sum_{u \neq v} d(v,u)}$$

Measures **reach** — high when average shortest-path distance to all other nodes is small. Captures how fast information spreads from this node.

Example: An emergency-dispatch hub close to all districts has high closeness — any location can be reached in minimum average time.

Speaker notes

Degree and closeness both measure "how well connected is this node" but in different senses. Degree is purely local: it counts immediate neighbors and ignores the rest of the network. Closeness requires knowing the entire graph (run BFS from every node) and measures how well a node is positioned to broadcast or collect information. A node can have high degree but low closeness if it sits in a dense peripheral cluster far from the rest of the graph.

Betweenness and Eigenvector Centrality

Betweenness centrality

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

Measures **control** — fraction of all shortest paths that pass through v. σ_{st}: number of shortest s–t paths; $\sigma_{st}(v)$: those passing through v.

Example: A router at a continental internet exchange has high betweenness — all intercontinental traffic passes through it.

Eigenvector centrality

$$C_E(v) = \frac{1}{\lambda} \sum_u A_{vu}, C_E(u)$$

Measures **prestige** — recursive, a node is important if connected to other important nodes. A: adjacency matrix; λ: its leading eigenvalue.

PageRank (Google, 1998) is the directed, damped variant: prestige flows along directed citation/link edges, with a damping factor to handle dangling nodes.

Speaker notes

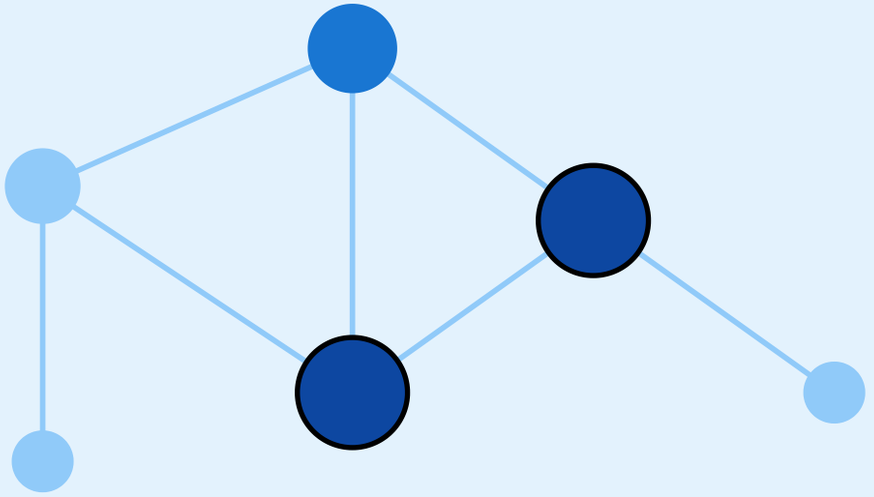
Betweenness is the most computationally demanding of the four: computing it exactly requires running shortest-path BFS from every node, giving $O(|V| \cdot |E|)$ for unweighted graphs (Brandes, 2001). Eigenvector centrality is defined by a fixed-point equation — the centrality vector is the leading eigenvector of the adjacency matrix, computed by power iteration. Both answer different questions from degree and closeness: betweenness captures *control* over information flow (who can act as gatekeeper?), eigenvector captures *prestige* (whose opinion is amplified by the audience of their audience?).

Same Graph, Four Views of Importance

Same graph, four notions of "important"

Degree

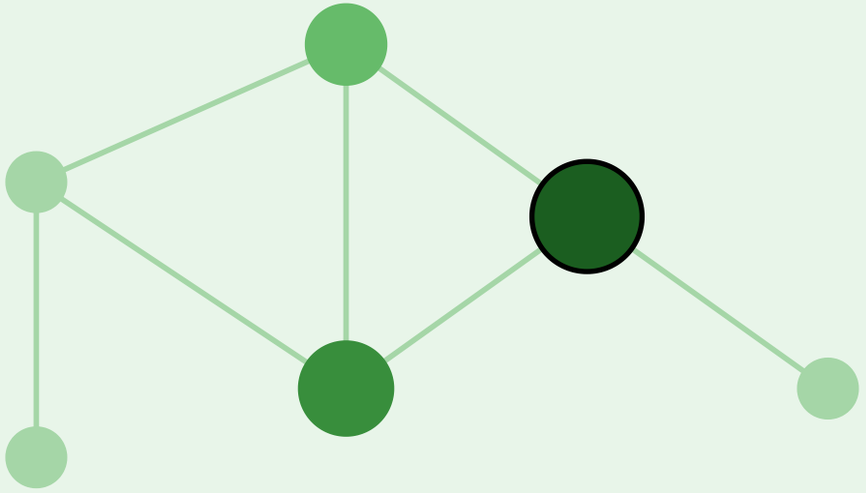
how many direct connections



node size ∝ degree

Closeness

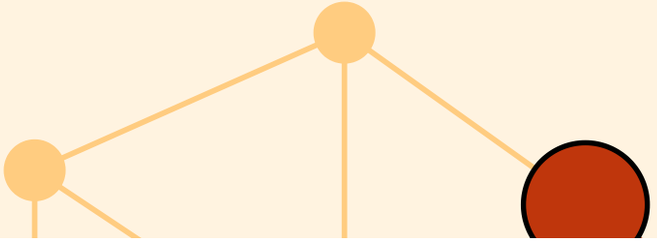
low average distance to all others



closer to center → larger

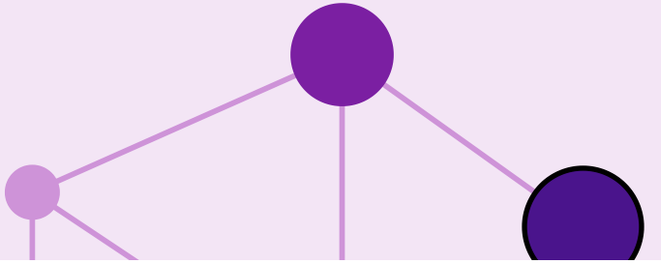
Betweenness

how often lies on shortest paths



Eigenvector

connected to other important nodes



Node size in each panel encodes that panel's centrality value. The same graph yields different "most important" nodes depending on which measure is used — confirming that centrality is always a theory of importance, not a neutral measurement.

Speaker notes

This visualization makes the central conceptual point of the module concrete: there is no single answer to "which node is most important". The bridge node dominates betweenness but not eigenvector; the densely embedded node dominates eigenvector but not betweenness. Choosing a centrality measure is choosing a theory about what kind of influence matters in the system being analyzed.

Back to the Workplace

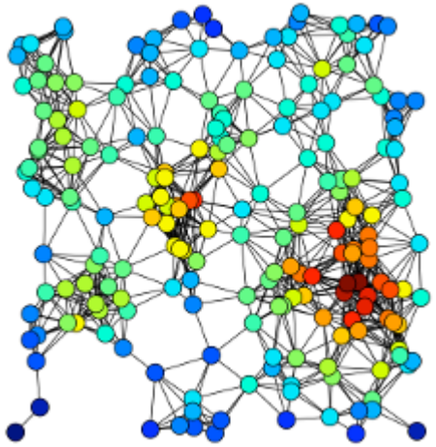
In the workplace figure, each measure picks a different winner:

- **Degree** → Ana (or Ben): most direct contacts inside team A's core
- **Closeness** → Dia: smallest average distance to everyone, because she sits on the only bridge
- **Betweenness** → Dia *and* Fin: every shortest path between the teams passes through both
- **Eigenvector** → Ana or Ben: connected to densely-connected others, which amplifies recursively

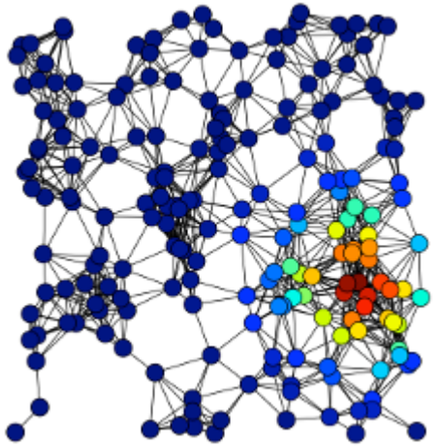
Speaker notes

Running through the four centralities on the same small concrete graph makes the pluralism tangible. The crucial insight: the "most important" answer depends on which theory of importance you adopt. Dia wins on betweenness because she is the only broker, but she loses on eigenvector because team B's neighbors are less densely connected than team A's. In the workplace figure there is no single "most important" node — there is only a most important node *for a given question*.

Real-World Examples



Source: Easley & Kleinberg 2010



Source: Easley & Kleinberg 2010

Left: Degree highlights raw connection counts. **Right:** Eigenvector centrality refines this — a node is important if it links to other important nodes. Google's PageRank is the directed variant of eigenvector centrality applied to the Web.

Takeaway

Centrality is always tied to a theory of influence, access, or control. Choosing a measure is choosing a theory — not collecting an objective fact.

Quick Check

For each scenario, decide which centrality measure is most appropriate and justify briefly.

1. You want to identify the most influential gossip in a small village.
2. You want to find emergency-response hubs that can reach all districts within minimum time.
3. You want to detect which router, if attacked, would slow internet traffic between continents most.
4. You want to find the most prestigious researcher in a citation network.

Speaker notes

The four scenarios are deliberately matched one-to-one with the four measures. Students who answer "degree" or "betweenness" to all four miss the conceptual point: each scenario embeds a different theory of what "important" means, and the right measure is the one that matches the theory.

Quick Check — Solution

1. **Degree centrality** — gossip is local, direct contacts matter.
2. **Closeness centrality** — minimizing average distance to all districts.
3. **Betweenness centrality** — the router's role is to lie on paths between continents.
4. **Eigenvector centrality / PageRank** — prestige flows from being cited by others who are themselves prestigious.

What Counts as a Community?

Guiding Question: Can we give "community" a mathematical definition precise enough that an algorithm could find one?

Speaker notes

Communities are intuitively obvious in a drawing but surprisingly hard to define formally. The definition determines what algorithms can detect and what they will miss. This module introduces modularity as the canonical answer — and then shows that the canonical answer is computationally out of reach, exactly as MaxSTC was in Lecture 03.

Working Definition

Community (informal). A subset of nodes $S \subseteq V$ with

- relatively many edges *inside* S
- relatively few edges *leaving* S
- density meaningfully higher than the surrounding graph.

Different formal definitions instantiate this intuition differently. The most influential is **modularity**, which compares observed internal density against a random null model.

Speaker notes

The informal definition is intentional: it gives the structural criterion without committing to a specific objective function. Modularity, conductance, and density-based definitions all satisfy the informal criterion but operationalize it differently. The pluralism is a feature — different applications need different sharpnesses of community boundary — but for the rest of the lecture we focus on modularity because it is the single most widely used objective.

Modularity — The Canonical Objective

Modularity Q of a partition $C_1, \ldots, C_k$ of a graph with m edges and adjacency matrix A:

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

where k_i is the degree of node i and $\delta(c_i, c_j) = 1$ if i and j are in the same community, else 0.

Interpretation. For every within-community pair, compare the *observed* edge A_{ij} to the *expected* edge density $k_i k_j / 2m$ under a random rewiring that preserves degrees (the **configuration model**). Q is the total surplus of within-community edges over chance.

Q ranges roughly in $[-\frac{1}{2}, 1]$. Values in $[0.3, 0.7]$ typically indicate significant community structure; $Q \approx 0$ means no more internal density than expected by chance.

Speaker notes

Modularity is the single most important concept in this lecture. It converts the vague notion of "community" into a number we can optimize. The configuration-model null is what makes it statistical rather than arbitrary: instead of asking "are there many edges inside this group?" it asks "are there *more* edges inside this group than we would expect if we kept each node's degree but rewired everything at random?". That reframing is precisely why modularity can distinguish real communities from high-degree hubs whose connections merely happen to concentrate.

The Catch — Modularity Maximization Is NP-Hard

Result (Brandes et al., 2008). Finding the partition that maximizes Q on a general graph is **NP-hard**. No polynomial-time algorithm for the exact optimum is known.

Speaker notes

This is the slide where the L03–L04 narrative arc closes. In L03, we had a clean theoretical construct (STC labeling) whose exact recovery (MaxSTC) was NP-hard, so we fell back on clustering coefficient and neighborhood overlap as polynomial-time proxies. In L04, we have a clean theoretical construct (modularity-optimal partition) whose exact recovery is also NP-hard, and we will fall back on Girvan–Newman and Louvain as polynomial-time proxies. Same shape, different level of the graph.

The Catch — Modularity Maximization Is NP-Hard

> **Result (Brandes et al., 2008).** Finding the partition that maximizes Q on a general graph is **NP-hard**. No polynomial-time algorithm for the exact optimum is known.

Sound familiar? This is the exact same situation as **MaxSTC** in Lecture 03: a theoretically clean objective that is computationally out of reach on real data.

The Catch — Modularity Maximization Is NP-Hard

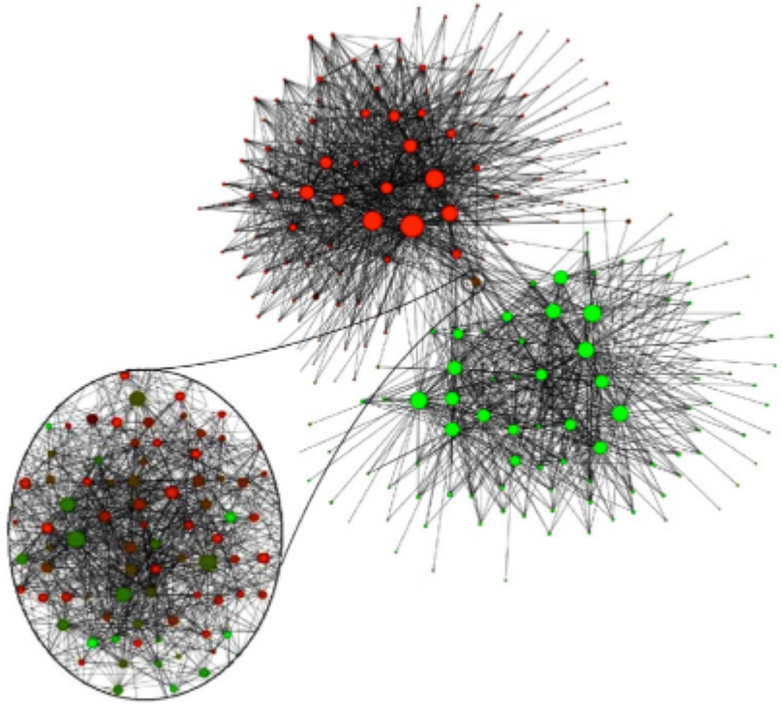
> **Result (Brandes et al., 2008).** Finding the partition that maximizes Q on a general graph is **NP-hard**. No polynomial-time algorithm for the exact optimum is known.

Sound familiar? This is the exact same situation as **MaxSTC** in Lecture 03: a theoretically clean objective that is computationally out of reach on real data.

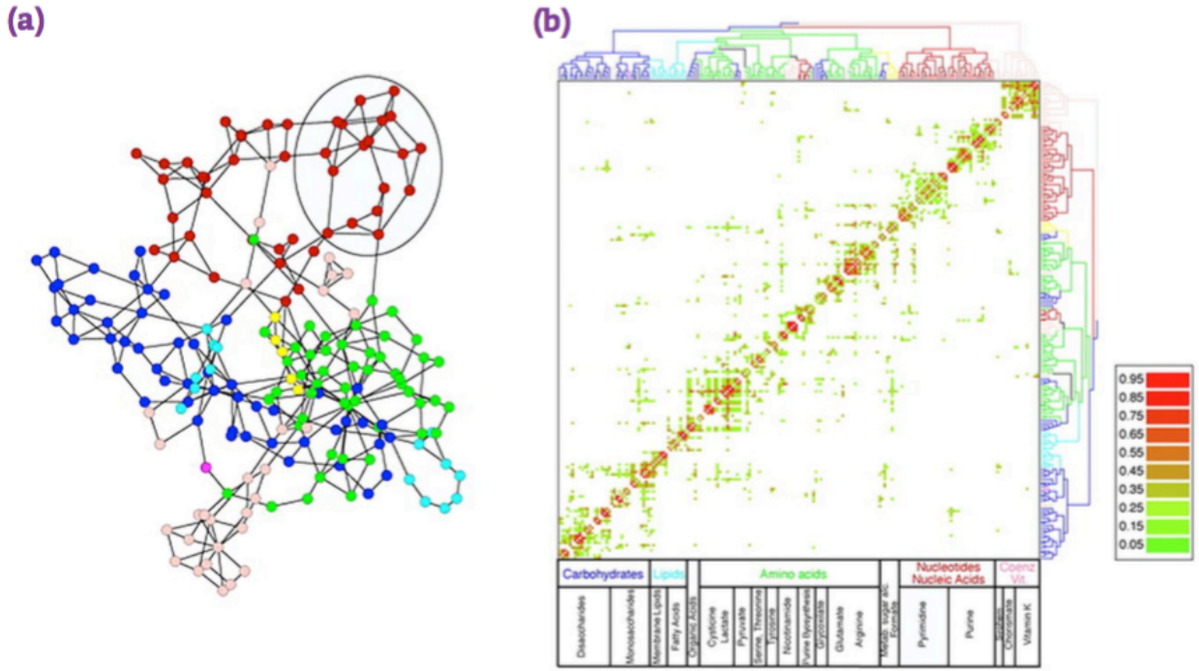
Consequence. The rest of the module introduces two families of polynomial-time *heuristics* that find partitions with high Q without guaranteeing the global maximum:

- **divisive** (Girvan–Newman): peel apart high-betweenness edges
- **agglomerative** (Louvain, Leiden): greedily merge nodes that increase Q

Example Cases



Source: Easley & Kleinberg 2010



Source: Easley & Kleinberg 2010

Three different domains, the same structural pattern: dense internal connectivity and sparse cross-group connectivity. The Belgium phone graph reveals the linguistic divide between Flemish and Walloon speakers — an external label the algorithm was never told. Metabolic communities reflect functional pathway groupings.

Speaker notes

The cross-domain examples make the universality argument: the same modularity-based tools recover meaningful groups in social, biological, and technical networks. The Belgium example is particularly striking because the linguistic boundary was never an explicit feature of the data — it was discovered from call patterns alone by a Louvain-style modularity maximizer.

Takeaway

Modularity Q is the canonical mathematical definition of "community": excess internal density over a random-graph null model. Maximizing it exactly is NP-hard — so every practical community detection method is a heuristic.

Quick Check

A graph has two dense clusters of 50 nodes each connected by a single edge, plus one small triangle attached to one cluster via a single edge.

1. Roughly how many natural communities does this graph have?
2. Will modularity maximization find all of them?
3. If you asked a balanced 2-partition algorithm to split it into two equal halves, what might go wrong?

Speaker notes

The graph has three natural communities, and modularity maximization should find them on a graph this size — but there is a subtle trap. Modularity has a well-known *resolution limit* (Fortunato & Barthélemy 2007): in large graphs, modularity cannot detect communities smaller than roughly $\sqrt{2m}$ edges. On the small example above the triangle is easy to find, but if the two big clusters grew to 5000 nodes each, the triangle would disappear below the resolution threshold. The balanced partition question is a separate trap: enforcing equal sizes forces the algorithm to either cut through a real community or merge the triangle with the wrong neighbor. Either way, a structural constraint hides real structure.

Quick Check — Solution

1. Three natural communities — the two dense clusters and the small triangle.

2. Modularity will find them on this small graph, but in large graphs modularity has a **resolution limit**: it systematically fails to detect communities smaller than roughly $\sqrt{2m}$ edges. Scale the two clusters up to 5000 nodes each and the triangle vanishes from view.

3. Balanced 2-partition is forced to produce equal-sized groups. It will either cut through one of the dense clusters or merge the triangle with the wrong cluster. The constraint distorts the answer.

Finding Communities — Divisive and Agglomerative

Guiding Question: If modularity maximization is NP-hard, how do we find good partitions in practice

Speaker notes

This module collects the practical heuristics used to approximate modularity maximization. Two broad families: divisive (Girvan–Newman) works top-down by removing inter-community edges; agglomerative (Louvain, Leiden) works bottom-up by merging nodes that increase modularity. Both produce hierarchies, and both are chosen over exact optimization because they are polynomial-time.

Two Strategies

Divisive (top-down). Start with the whole graph as one cluster. Repeatedly remove edges that lie *between* communities, peeling the graph apart. Example: Girvan–Newman.

Agglomerative (bottom-up). Start with each node as its own cluster. Repeatedly merge the pair whose merge most *increases* modularity. Example: Louvain, Leiden.

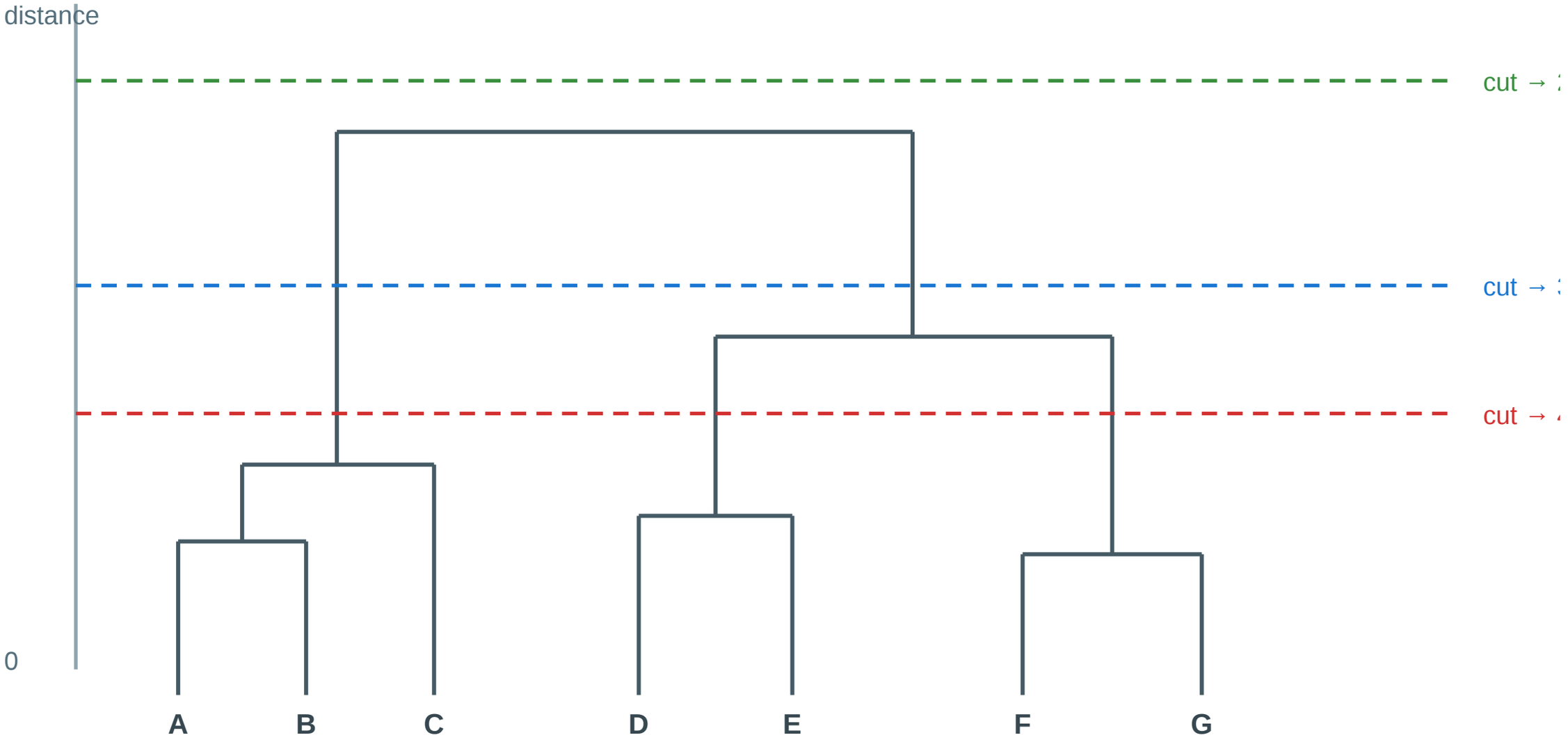
Both strategies produce a hierarchy — a **dendrogram** — rather than a single flat partition. Where to cut the dendrogram is the analyst's modeling decision.

Speaker notes

Divisive methods target the inter-community edges directly, which aligns with the bridge intuition from Lecture 03 — inter-community edges carry many shortest paths, so they have high edge-betweenness. Agglomerative methods are cheaper and currently dominate practice (Louvain is the default in NetworkX and Gephi, Leiden is the modern improvement). Both families produce dendrograms, so the module-level takeaway is the same: algorithms give you a hierarchy; you, the analyst, decide where to cut.

The Dendrogram as Output

Hierarchical clustering: the cutoff is a modeling choice



Each horizontal cut yields a different clustering — the analyst chooses the level

Every agglomerative or divisive run produces a hierarchy like this. Each horizontal cut is a valid clustering; the "right" one depends on the downstream task. Picking the cut with the highest modularity Q is the default rule.

Speaker notes

The dendrogram is the explicit representation of multi-scale community structure. Cutting it at different heights produces different partitions, and each cut is structurally valid. The decision about where to cut is necessarily external to the algorithm — it depends on what the communities will be used for, or on picking the height that maximizes Q.

Girvan–Newman — The Divisive Algorithm

Algorithm: Girvan–Newman

(Input: Graph $G = (V, E)$*)*

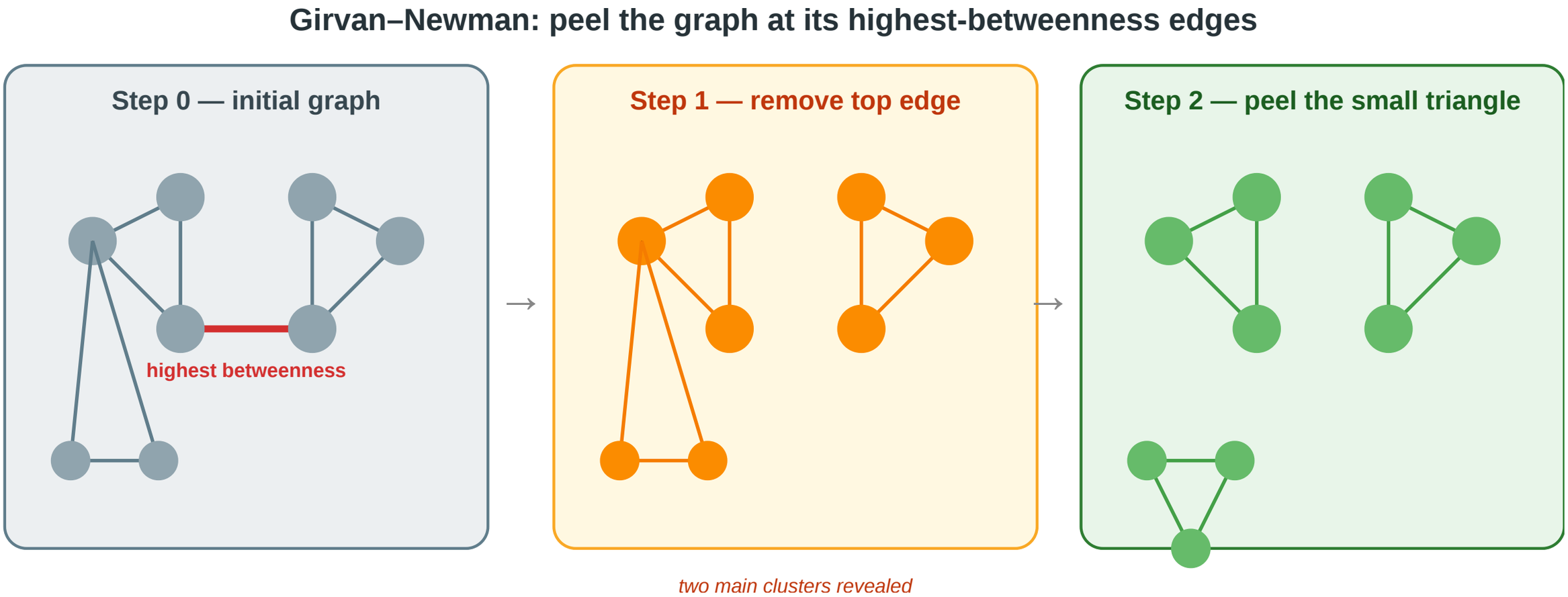
1. Compute the **edge betweenness** of every edge — the fraction of all shortest paths that pass through it.
2. Remove the edge with the **highest** edge betweenness.
3. Recompute edge betweenness on the remaining graph.
4. Repeat until no edges remain — record the partition at each step.
5. Choose the partition that maximizes **modularity** Q.

Edges with high edge-betweenness are exactly the **local bridges** and **weak ties** from Lecture 03 — they carry many shortest paths because all cross-community communication must cross them. Girvan–Newman is an algorithmic version of Granovetter's weak-tie argument.

Speaker notes

The recompute step in line 3 is what makes Girvan–Newman conceptually clean but computationally expensive: edge betweenness must be recalculated after every removal because removing one edge changes the shortest paths used by all the others. The whole algorithm runs in $O(|V| \cdot |E|^2)$ worst case — fine for teaching, impractical for graphs beyond a few thousand nodes. This is why modern practice uses Louvain / Leiden instead.

Worked Example



Step 0: the bridge edge has the highest betweenness because every path between the two clusters must cross it. **Step 1:** removing it reveals two main clusters but the small triangle remains weakly attached. **Step 2:** the next-highest-betweenness edge is the link to the triangle — removing it isolates the third community. Final partition: three communities.

Speaker notes

The worked example shows the algorithm's two characteristic moves: detaching a major community boundary, then peeling smaller substructures. Notice that edge-betweenness recomputation is essential — after step 1, the highest-betweenness edge changes because the graph topology has changed.

Louvain and Leiden — Modern Practice

Louvain (Blondel et al., 2008). Agglomerative modularity maximizer.

Phase 1: For each node, try moving it to each neighbor's community; keep the move that most increases Q. Repeat until no move improves Q.

Phase 2: Contract each community into a super-node and repeat Phase 1 on the new graph. Iterate until Q converges.

Leiden (Traag et al., 2019). Fixes a subtle Louvain flaw (disconnected communities) and guarantees well-connected output at each level.

Why these dominate practice. Near-linear time in $|E|$; handle millions of nodes; directly optimize Q; no edge-betweenness recomputation.

Speaker notes

Louvain is currently the default community-detection algorithm in NetworkX, Gephi, and most network-analysis packages — it scales to graphs orders of magnitude larger than Girvan–Newman. Leiden is the 2019 improvement that fixed a structural bug in Louvain (it could return communities that were internally disconnected) and is now the recommended default in research. Both directly optimize modularity greedily without touching edge betweenness.

Graph Partitioning — Cut-Based Methods

Min-cut / Max-flow (Ford–Fulkerson) Find the smallest set of edges whose removal disconnects two specified groups. Exact in polynomial time — but minimizes raw cut size, not normalized density.

Kernighan–Lin (1970) Local search: initialize a balanced 2-partition at random; iteratively swap node pairs to decrease the cut size. Fast in practice; converges to a local optimum.

Spectral partitioning Compute the Laplacian $L = D - A$ of the graph. The **Fiedler vector** (eigenvector of the second-smallest eigenvalue λ_2) encodes the natural 2-partition: nodes with positive entries go left, negative entries go right.

λ_2 is the **algebraic connectivity**: $\lambda_2 = 0$ means the graph is already disconnected. Spectral methods generalize to k communities by using the k smallest eigenvectors — the basis of **spectral clustering**.

Speaker notes

Cut-based partitioning predates modularity and remains important in chip design, load balancing, and parallel computing — contexts where the exact number of parts is fixed and balance matters more than organic community structure. Min-cut gives the theoretically tight answer for two specific groups but is sensitive to hub nodes (a single high-degree hub connected to both sides will dominate the cut). Kernighan–Lin is the practical engineering heuristic for VLSI layout. Spectral partitioning bridges graph theory and linear algebra: the Fiedler vector carries the global geometry of the graph into a 1D projection that can be thresholded to produce a partition. The connection to graph Laplacians also connects directly to graph neural networks — the spectral view is the foundation of GCNs.

Embedding-Based Community Detection

Node embeddings + clustering

node2vec (Grover & Leskovec, 2016) Random-walk–based embedding: simulate biased random walks on the graph, then train Word2Vec–style skip-gram to produce a low-dimensional vector per node. Communities emerge as clusters in embedding space.

Pipeline: node2vec → k-means / DBSCAN → community labels

Hyperparameter p controls return probability (DFS-like, captures communities), q controls exploration (BFS-like, captures roles). Setting $p \gg q$ favors dense cluster detection.

Graph Neural Networks

GNN-based community detection Train a GNN encoder (e.g. GCN, GraphSAGE) to produce embeddings that maximize an unsupervised objective — modularity, contrastive loss, or cluster assignment entropy. The decoder maps embeddings to community memberships.

Approach	Key idea
GRAPH-BERT	Self-supervised pre-training on subgraphs
DMGI	Contrastive multiplex embeddings
DMoN	Differentiable modularity as loss

GNNs naturally handle **overlapping communities** (soft cluster assignment) and **attributed graphs** (node features + topology jointly).

Speaker notes

Embedding-based community detection represents the current research frontier. node2vec is now a standard baseline — available in the PyTorch Geometric and NetworkX ecosystems — and runs efficiently on graphs with millions of nodes. The intuition is straightforward: if two nodes co-occur frequently in random walks, they are structurally similar and their embeddings will be nearby; k-means on those embeddings recovers communities. GNN-based methods are more powerful because they can use node and edge features alongside topology, handle soft (overlapping) community membership, and be trained end-to-end with application-specific objectives. The key limitation of all embedding approaches: the embedding dimension and clustering hyperparameters (k, distance threshold) must be chosen before running, adding new modeling decisions that modularity-based methods avoid.

Algorithm Comparison

Method	Strategy	Complexity	Strength	Limitation
Min-cut	Cut minimization	$O(\|V\| \cdot \|E\|)$	Exact for 2-partition	Ignores density, favors small cut
Kernighan–Lin	Local swap	$O(\|V\|^2 \log \|V\|)$	Fast for balanced partition	Local optimum, fixed k
Spectral	Laplacian eigenvectors	$O(\|V\|^3)$ full / faster approx	Global structure, principled	Slow exact; needs k in advance
Girvan–Newman	Edge betweenness	$O(\|V\| \cdot \|E\|^2)$	Hierarchy, no k needed	Too slow for large graphs
Louvain / Leiden	Modularity greedy	$O(\|E\|)$ approx	Fast, scalable, no k needed	Resolution limit, local opt
node2vec + clustering	Random-walk embed	$O(\|E\|)$ walks + embed	Scales, uses node features	Needs k, walk hyperparams

Method	Strategy	Complexity	Strength	Limitation
GNN-based	Neural + graph structure	Varies	Soft membership, features	Requires training data or objective design

Speaker notes

This table is the key synthesis slide for the module. Students should be able to match each method to the right use case: Louvain/Leiden for exploratory analysis of large graphs with no prior on k; spectral clustering for medium-sized graphs where you want a principled 2–5 partition; node2vec for graphs with rich structure where embedding locality is meaningful; GNN-based when you have node features or need overlapping communities. The complexity column signals when each method becomes impractical — Girvan–Newman is pedagogically essential but computationally obsolete above ~10k nodes.

Takeaway

Divisive and agglomerative heuristics approximate the NP-hard modularity optimum in polynomial time. Girvan–Newman is the didactic divisive method; Louvain and Leiden are the scalable agglomerative methods used in practice. Graph partitioning (min-cut, spectral) targets fixed-k balanced splits; embedding-based methods (node2vec, GNNs) detect communities from representation space and can handle overlapping memberships and node features.

Quick Check

You run Girvan–Newman on a small graph and get a dendrogram with a maximum Q at the cut that produces 4 communities. You then run Louvain on the same graph and it reports 3 communities with a slightly lower Q.

1. Which answer is "correct"?
2. What could explain the discrepancy?

Speaker notes

Neither is correct in an absolute sense — both are heuristics approximating an NP-hard optimum. Girvan–Newman's divisive peeling can find finer structure because it tests every possible cut, but its higher Q might be fragile to noise. Louvain's greedy merges can get stuck in local optima and miss small communities. The honest answer: both are reasonable, and the discrepancy itself is evidence that the solution is not unique — the student should report both and discuss the tradeoff.

Quick Check — Solution

1. Neither is absolutely correct — both are heuristics approximating the NP-hard modularity optimum. The higher Q value is weak evidence, not proof, and the difference may be within noise.

2. Possible explanations:

- Louvain is greedy and can get stuck in local optima that miss small communities.
- Girvan–Newman explores more cuts but is sensitive to the order of edge removals.
- Modularity itself has a resolution limit; both methods share this blind spot.
- The graph may genuinely admit multiple near-optimal partitions — real structure can be ambiguous.

The honest practice is to run multiple algorithms, report agreement, and treat disagreement as a signal about the graph itself.

Empirical Anchor — Zachary's Karate Club

Guiding Question: Do community-detection heuristics actually recover real social groups — and how do we know?

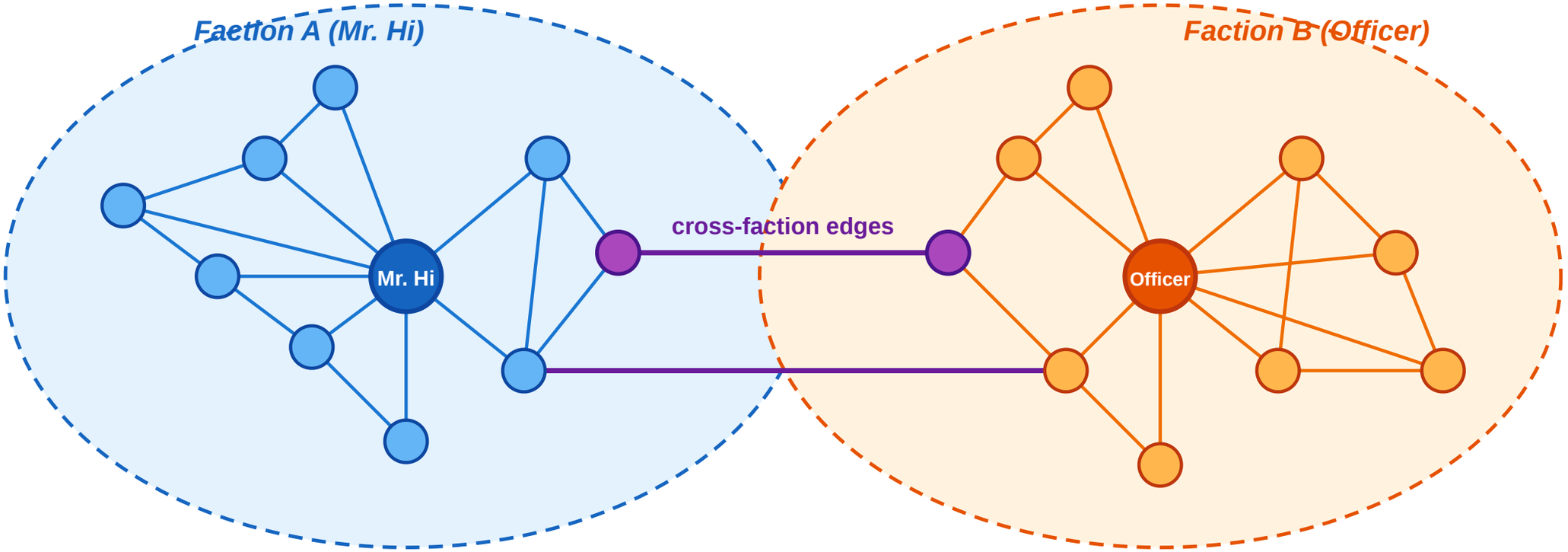
Speaker notes

Zachary's karate club is the canonical validation dataset for community detection. It is a small, real social network with a documented ground-truth split, which makes it one of the very few graphs where we can actually check whether an algorithm "got the right answer". It has become so iconic in network science that the first speaker at a conference to mention it in a talk traditionally wins the "Zachary Karate Club Club" prize.

The Story

Zachary's Karate Club — the community-detection benchmark

34 members · pre-split friendships · two factions formed around Mr. Hi and the Officer



hub (instructor / admin) · regular members · ambiguous (on faction boundary) · cross-faction tie

Schematic — the real Zachary graph has 34 nodes and 78 edges; the split is the documented outcome.

Zachary (1977), *An Information Flow Model for Conflict and Fission in Small Groups*, **J. Anthropological Research 33, 452–473**.

Wayne Zachary observed a university karate club for two years. A conflict arose between the instructor (**Mr. Hi**) and the chief administrator (**Officer**), and the club eventually **split in two** — with each member following one or the other.

Speaker notes

The crucial detail: Zachary recorded the network of *friendships* between members *before* the split happened. The ground truth (who joined which faction) was the natural outcome of the conflict, not an analyst's label. That means we can run any community-detection algorithm on the pre-split graph and directly check how many members it assigns to the correct faction.

What the Algorithms Find

Method	Typical result
Girvan–Newman	Recovers the two factions with **1 misclassified** node
Louvain / Leiden	Recovers the two factions; typically finds **4 sub-groups** at max Q
Max-modularity (exact on this small graph)	Finds the same 4 sub-groups nested inside the 2 factions

The ~1-node misclassification is structurally meaningful: the node in question genuinely sat on the boundary between the two factions during the conflict period, and which side they joined was a close call even to Zachary himself.

Speaker notes

The karate-club result is a success story and a cautionary tale at the same time. Success: the heuristics find the real split from the friendship network alone, which is strong evidence that community structure in social networks is a real phenomenon and not a statistical mirage. Cautionary: the exact modularity maximum finds four sub-groups, not two — modularity's "best" answer disagrees with the ground-truth binary split, because internally the two factions each contain two tightly-connected sub-cliques. The algorithm is not wrong; it is answering a slightly different question.

What the Study Could Not See

Even in this iconic success case, the empirical validation has blind spots that parallel Lecture 03's gap between construct and measurement:

1. Modularity resolution limit (Fortunato & Barthélemy 2007). On the 34-node karate graph the exact max-Q partition has **4** communities, not **2**. The ground-truth split is not even the modularity optimum — the algorithm is answering "which partition maximizes Q", not "which partition matches the real factions".

2. Binary ground truth. Zachary's data records which faction each member *eventually* joined — but not the *strength* of their preference, their uncertainty during the conflict, or whether a third "neutral" group briefly existed. The true social structure may have been continuous or overlapping.

3. Static snapshot. The friendships were recorded at a single time. The dynamics of the conflict — who influenced whom, when someone shifted — are invisible to the algorithm.

Speaker notes

This slide is the L04 analogue of "What the Kossinets-Watts Study Could Not See" in L03. The parallel is intentional: in L03 the binary email data hid tie strength; in L04 the binary faction label hides community overlap and dynamics. Both are reminders that the empirical benchmark is always narrower than the theoretical construct we are trying to validate.

Takeaway

Zachary's karate club shows that community-detection heuristics recover real social structure from graph topology alone — but also that even the "canonical" answer has blind spots: modularity's resolution limit, binary ground truth, and static snapshots all hide real structure the algorithm cannot see.

Wrap-Up — Closing the Modeling Loop

Across Lectures 03 and 04 we have traversed the full modeling/measurement loop twice:

Level	Theoretical ideal	NP-hard recovery	Polynomial proxy	Empirical test
Edges (L03)	Strong/weak labeling	MaxSTC	Clustering coeff., overlap	Kossinets–Watts email
Nodes / Groups (L04)	Modularity-optimal partition	Modularity maximization	Girvan–Newman, Louvain, Leiden	Zachary karate club

Every concept in the two lectures occupies one of those four cells.

Reading

Chapter 3.6 and Chapter 9 of Easley & Kleinberg (2010) cover community structure, brokerage, and centrality. Newman (2010), Chapter 7, gives the formal treatment of centrality measures. For modularity and Louvain, see Blondel et al. (2008) and Traag et al. (2019) for Leiden.

Speaker notes

The closing table is the summary of the L03–L04 arc: two levels of structural analysis, same four-step loop. Lecture 05 will break this pattern by asking a different kind of question: instead of modeling structure, it asks *why* structure forms — when similarity causes friendship, when friendship causes similarity, and how social contexts shape the networks that emerge from them. Homophily, selection, and affiliation networks come next.

Image Credits & References

Easley, David and Kleinberg, Jon (2010). Networks, Crowds, and Markets: Reasoning About a Highly Connected World. *Cambridge University Press*. https://www.cs.cornell.edu/home/kleinber/networks-book/